

一些关于Openpilot资料汇总

赞



傅里叶的魔豆

e^{iπ} + 1 = 0

+ 关注他

41 人赞同了该文章

注：本文仅仅是对一些公开的资料汇总，方便大家学习交流，并无他意。如果侵权，联系小编，第一时间删除。

一、Openpilot开源项目+简介

Openpilot是开源的L2级别的自动驾驶辅助+开源系统，基于 android，主要用 python语言组织机构，c/c++核心计算，carla工具实现仿真，cereal为消息中间件+。Openpilot的实现原理类似于特斯拉靠的是纯视觉的解决方案，但因为camera只有两颗（一颗用于拍摄实现的路况，另一颗用于监控驾驶员），而不是像特斯拉有多颗摄像头（侧面及后方的），可以实现更多功能，如自动泊车等。所以，openpilot和主流ADAS辅助驾驶方案对比，优势可能主要在软件（模型）优化上，会比传统厂商做的更细腻。



github 全部项目（97个），可以主要分为以下几大类：自动驾驶汽车通讯、android系统、机器学习、其他辅助等等。开源代码+目录结构如下图所示：


```
.
├── cereal                # The messaging spec and libs used for all logs
├── common                # Library like functionality we've developed here
├── docs                  # Documentation
├── opendbc               # Files showing how to interpret data from cars
├── panda                 # Code used to communicate on CAN
├── third_party            # External libraries
├── pyextra               # Extra python packages
├── selfdrive             # Code needed to drive the car
│   ├── assets            # Fonts, images, and sounds for UI
│   ├── athena            # Allows communication with the app
│   ├── boardd            # Daemon to talk to the board
│   ├── camerad           # Driver to capture images from the camera sensors
│   ├── car               # Car specific code to read states and control actuators
│   ├── common            # Shared C/C++ code for the daemons
│   ├── controls          # Planning and controls
│   ├── debug             # Tools to help you debug and do car ports
│   ├── locationd         # Precise localization and vehicle parameter estimation
│   ├── logcatd           # Android logcat as a service
│   ├── loggerd           # Logger and uploader of car data
│   ├── modeld            # Driving and monitoring model runners
│   ├── proclogd          # Logs information from proc
│   ├── sensord           # IMU interface code
│   ├── test              # Unit tests, system tests, and a car simulator
│   └── ui                # The UI
```

Openpilot，这个项目是comma 最主要的一个项目，在EON/android设备及汽车之间架起的一座桥梁。通过EON移动设备的摄像头识别道路信息，及各种传感器的数据，经过分析计算后向汽车发出各种加/减速、转变等指令，以达到自动驾驶的目的。所有的后台服务，主要代码在项目的selfdrive 目录。

panda/opendbcPanda，是comma基于 STM32F413 开发一款让EON和汽车通讯的硬件，基于 opendbc 的汽车 OBD 协议，可以通过不同的 DBC 定义，解析和发送的 OBD 信号，起到了数据转换和解析的网关作用。它最多支持3个CAN，2个LIN和1个GMLAN。同时还可以为手机充电，同时也可能过USB和计算机连接并通讯。

comma10k/2k19，深度学习数据集这两个项目是comma公司个人觉得仅次于 openpilot 最重要的项目之一，这决定了自动驾驶的安全性及演进。comma2k19，这是加利福尼亚州280高速公路上通勤时间超过33小时的数据集。这意味着在加利福尼亚州圣何塞+和旧金山之间20公里的高速公路上行驶的2019年路段各长1分钟。comma2k19是一个完全可复制且可伸缩的数据集。数据是使用逗号EON收集的，该EON的传感器类似于任何现代智能手机，包括路面摄像头，手机GPS，温度计和9轴IMU。此外，EON还可以捕获原始GNSS测量数据以及用逗号灰色熊猫汽车发送的所有CAN数据。

cereal/laika，主要是一些数据采集、分析和可视化相关。

android/eon，openpilot 代码运行在基于 android 6 (cm13) 系统的设备之上，现在仅支持三款硬件：乐视、一加及comma（1,2,3代）。系统编译基本是SCONS。

二、基本操作

1.项目下载

注：最新（截止2021-12-20）Release版本为0.8.12


```
git clone github.com/commaai/open...
cd openpilot
git submodule update --init --recursive
```

2.环境配置

```
openpilot/tools/ubuntu_setup.sh
复制 "export PYTHONPATH=$HOME/openpilot" 进入 ~/.bashrc
安装 nvidia drivers: nvidia-xxx/cuda10.0/cudnn7.6.5
安装 tensorflow+ 2.2 GPU版本
安装 OpenCL Driver
安装 OpenCV4 (ignore the Python part)
```

3.编译

```
USE_WEBCAM=1 scons -j$(nproc)
```

4.运行

```
cd ~/openpilot/selfdrive/manager PASSIVE=0 NOSENSOR=1 USE_WEBCAM=1 ./manager.py
```

5.仿真运行+

```
./start_carla.sh ./start_openpilot_docker.sh
```

三、Openpilot项目架构

为了使其对环境（其他汽车，车道分叉）做出反应，我们需要能够读取传感器数据，对数据进行操作并更新执行器，例如方向盘或油门。从此描述中，我们可以得出核心系统功能的非常简单的抽象实现：

```
while (true) {

    read sensor data

    compute adjustments using machine-learning model

    apply adjustments to actuators}
```

Openpilot 使用300多个Python文件+来运行该系统，这些文件分为多个子模块，依赖项+和多个硬件组件。从传感器输入到执行器输出的路径上的主要体系结构组件和过程，它们的职责，在组件内部或组件之间应用的`设计模式`+以及在此体系结构中遇到的折衷。openpilot不是单个的整体过程，而是协调的`过程集群`。OpenPilot能够优先处理某些流程，从而使其能够遵循自动驾驶所需的严格计时要求。

1.硬件

运行openpilot系统必须需要三块硬件：

- 运行NEOS系统的EON硬件
- 基于STM32F4的panda硬件
- 官方支持的汽车

Panda是EON与汽车之间的安全执行桥梁，它使用对功能安全性有高度支持的芯片以及不久将符合MISRA（已完成），ISO26262和SIL2要求的软件。EON运行了一套基于CM13（Android 6.0）的Android修改版，在该版本中，本文将涉及的所有进程都将运行。而且汽车显然是汽车，其中3个CAN的布置与汽车线束的配置正确（令人惊讶的是有多少制造商符合此规格）。开车后，我们找到了由我们的硬件团队维护的熊猫固件。由此，我们到达了EON，并开始了我们的软件之旅。您可以

在openpilot/selfdrive中找到这些守护程序。它们通过cereal共享一个指定的IPC格式的消息。所有的消息都是单一发布者且多个订阅的传递，其摘要使得可以使用多个后端。目前，我们支持ZMQ和自定义msgq。

2.传感器和执行器

boardd，这是 panda 固件的接收方。它使用libusb进行通信并将原始USB层通信解析为“ CAN”数据包。在灰熊猫和黑熊猫上，它还广播来自NEO M8的GPS数据包。

camerad，这是相机堆栈。所我所知，这是高通唯一公开的定制的相机实现（其他应该都是闭源的），它直接与内核对话。它可以同时捕获道路和驾驶员摄像机，并处理自动对焦和自动曝光。

sensord，其余的传感器在这里处理，包括陀螺仪，加速计，磁力计和灯光。GPS和高通原始GPS数据也在这里被处理得很好。

NEOS内核空间⁺，这是Linux内核，而Android则是一团糟。你会发现我们开源的内核在这里，并且我们的Android仓库的fork在这里。内核统一运行在OnePlus 3和LePro 3这两款运行骁龙820的手机上。

3.数据处理

modeld，主模型位于models/driving_model.dlc文件中，它从公路摄像机中拍摄一张照片，并回答“我应该往哪里驾驶汽车？”这一问题。它还接受一个需求输入，该输入可以命令模型采取行动，例如转弯或改变车道。这是许多逗号魔法发生的地方，它具有深远的时序性，并且以超出深度学习技术水平的方式和技巧进行训练。

Modeld还运行models/posenet.dlc中的posenet。它接收两个帧并在它们之间输出6自由度变换。它用于校准和完整性检查，并且未经特别神奇的方式培训。

monitoringd（位于modeld目录中），这是驾驶员监视模型运行程序。它使用models/monitoring_model.dlc中的模型跟踪您的头部姿势，眼睛位置和眼睛状态。这个服务运行在DSP上，以便不使用其他守护程序所需的CPU或GPU资源，从而为它提供了巨大的增长空间。

locationd/ubloxd（TBD），因此，现在可以进行定位了，但这并不是真正的本地化程序的最终目标。现在，它将在ubloxd中解析来自ublox的数据流，然后将其与posenet组合以获得稳定的偏航估计。

calibrationd，该模型采用已校准的帧，这意味着在模型甚至查看图片之前就已校正了偏航和俯仰。这很重要，因为用户可以通过各种方式安装其EON，并且校准会输出转换以使其规范化。

4.控制模块

Controlsd，这是驱动汽车的主要100hz循环。它从计划者那里得到一个计划，并构造实现该计划所需的CAN数据包。它还发布了carState，这是我们通用的汽车抽象。

plannerd（位于controls目录中），模型输出不足以驾驶汽车。它输出汽车的位置，但不知道如何将汽车送到那里。在计划程序中，我们运行3个基于ACADO的MPC控制回路，其中1个用于横向控制，而2个用于纵向控制。

radard（位于controls目录中），这会将雷达解析为RadarState数据包。汽车具有所有不同的雷达，这使它们规范化。

paramsd（位于controls目录中），这是基于汽车的参数的学习者，例如轮胎刚度，转向角偏移和转向比。

5.日志/应用/UI

loggerd，该守护程序预订所有套接字⁺和摄像机，并将它们写出到日志中。

uploader/deleter，记录数据后，我们必须将其存储到云中。但是并非所有数据都将其存储到云中，我们删除旧数据以确保始终有可用空间。就像真正的行车记录仪一样。

UI，这是主要的驾驶界面。这是一个2300行的混乱局面，需要重构，但确实可以工作。

注：UI及前端应用部分确实是openpilot的软肋，和整体技术水平差了一个档次，主要是因为UI是

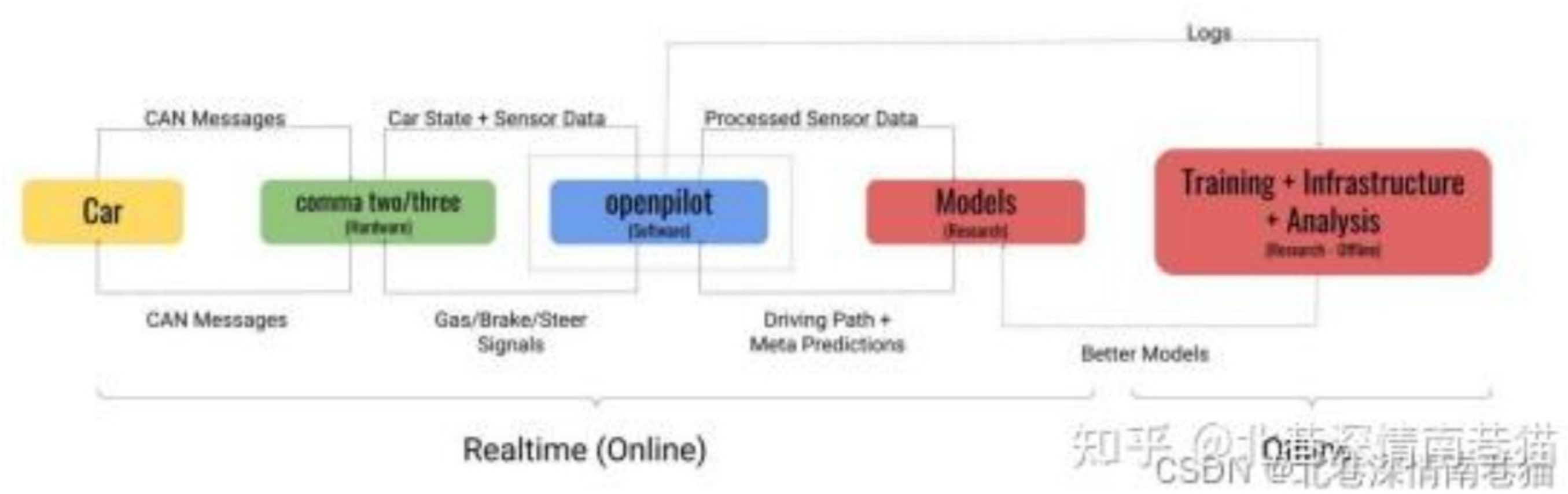
用c++写的，而不是用android原生界面库编写。
`apk.frame`，这是外边界。很快，它将被合并到C ++ UI中。其来源就在这里。
`apk.offroad`，这是设置菜单，入门教程，里程显示以及逗号白金版广告。它是用React Native编写的，并且源码在这里。
`athenad`，这项服务可让您与停放的汽车进行实时通信。查看API访问这里。

6.系统支持

`manager/thermald`，这将启动和停止使openpilot工作的进程群。
`updated`，该守护程序管理openpilot更新。
`logmessed/tombstoned/logcatd/proclgd`，这些是处理或系统行为异常时记录数据的助手。
`NEOS`用户空间，这是EON上基于termux的用户空间。它在Android上提供了类似Linux的环境。

7.数据流向框架

openpilot从各种传感器（摄像头、IMU、方向盘角度传感器、GNSS接收器等）读取数据，处理并且输入到大型神经网络。然后，它将神经网络的输出转换为可操作的命令，供汽车执行器执行。驾驶员疲劳检测，这也是openpilot的一项重要功能。



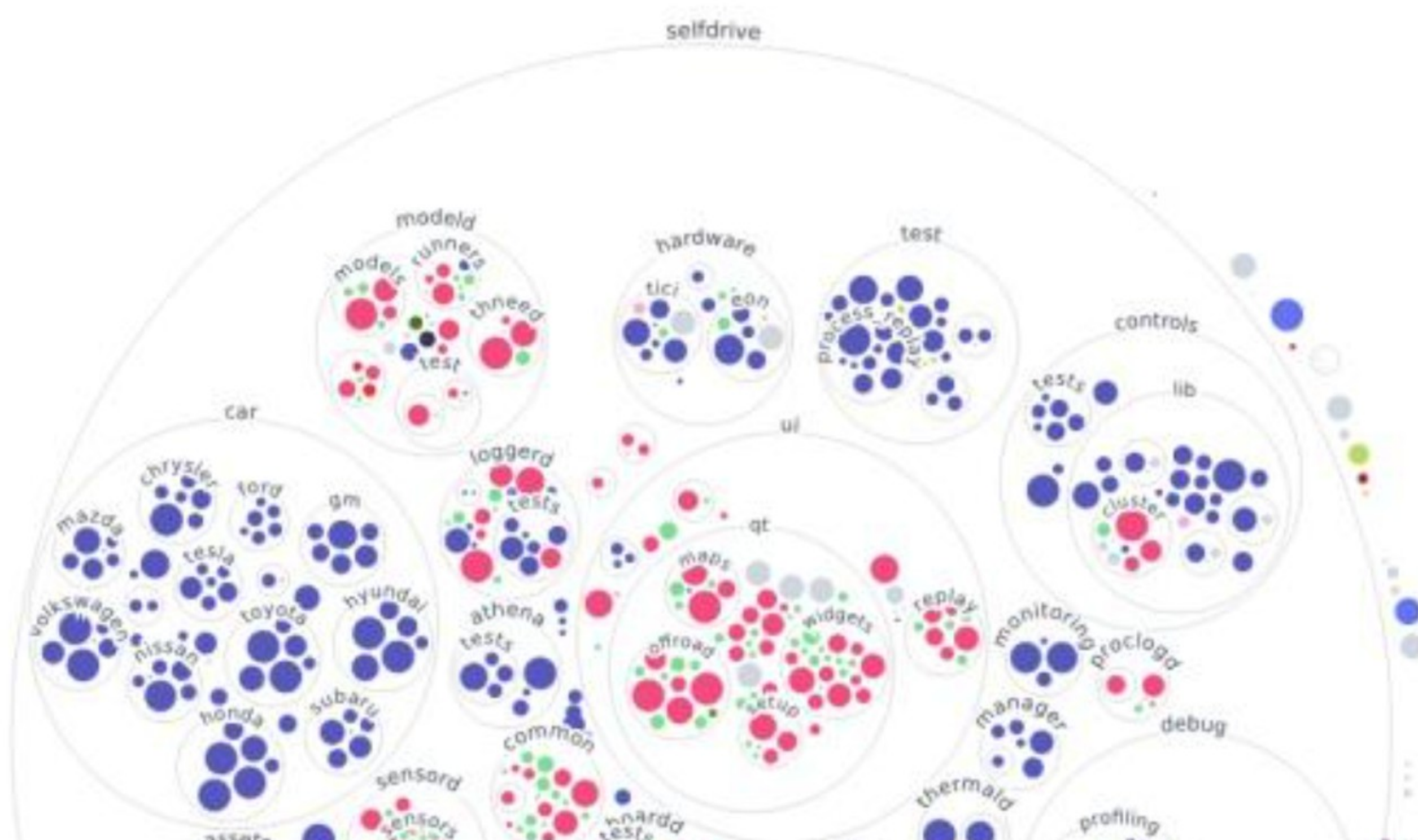
8.消息通信框架

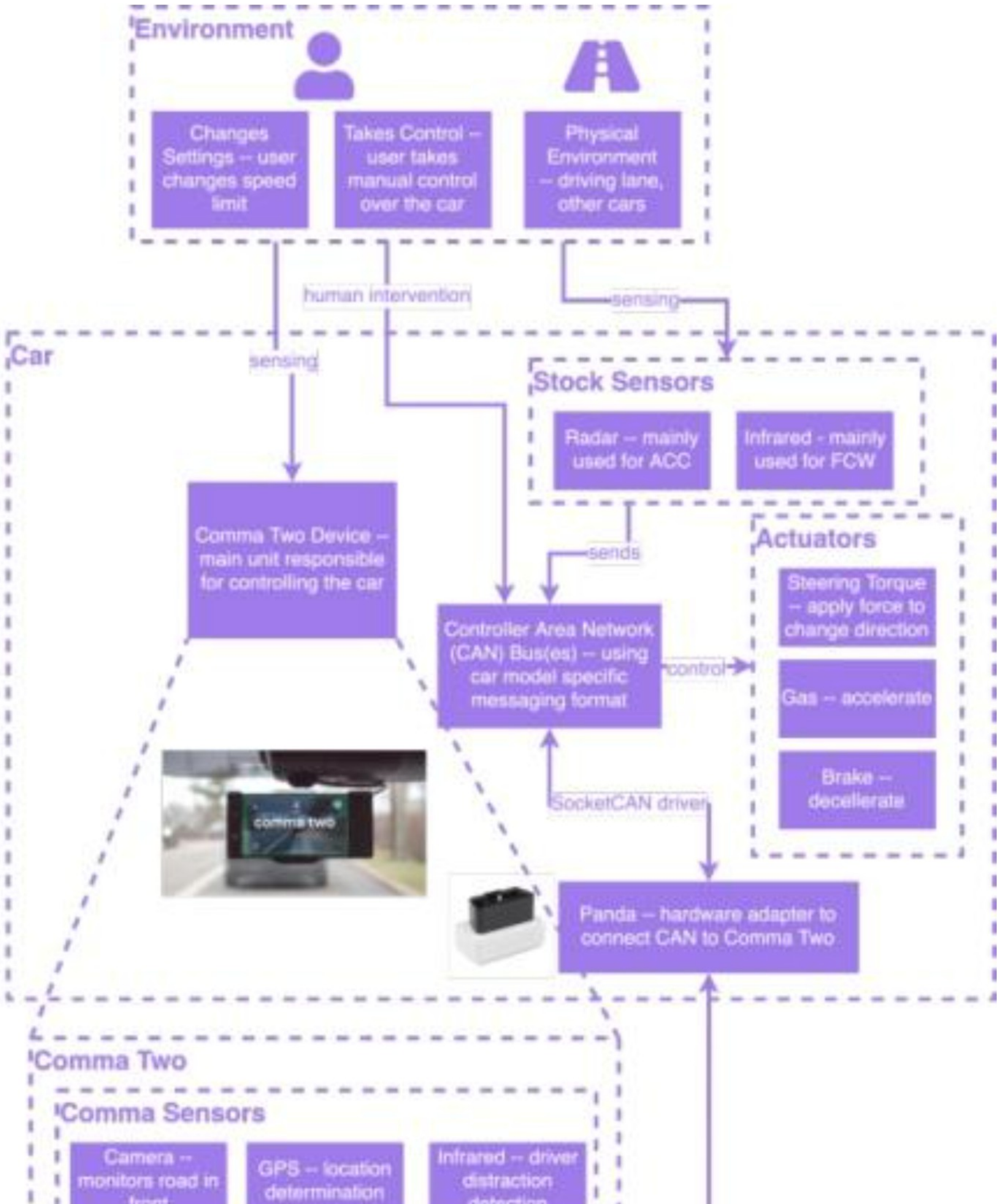
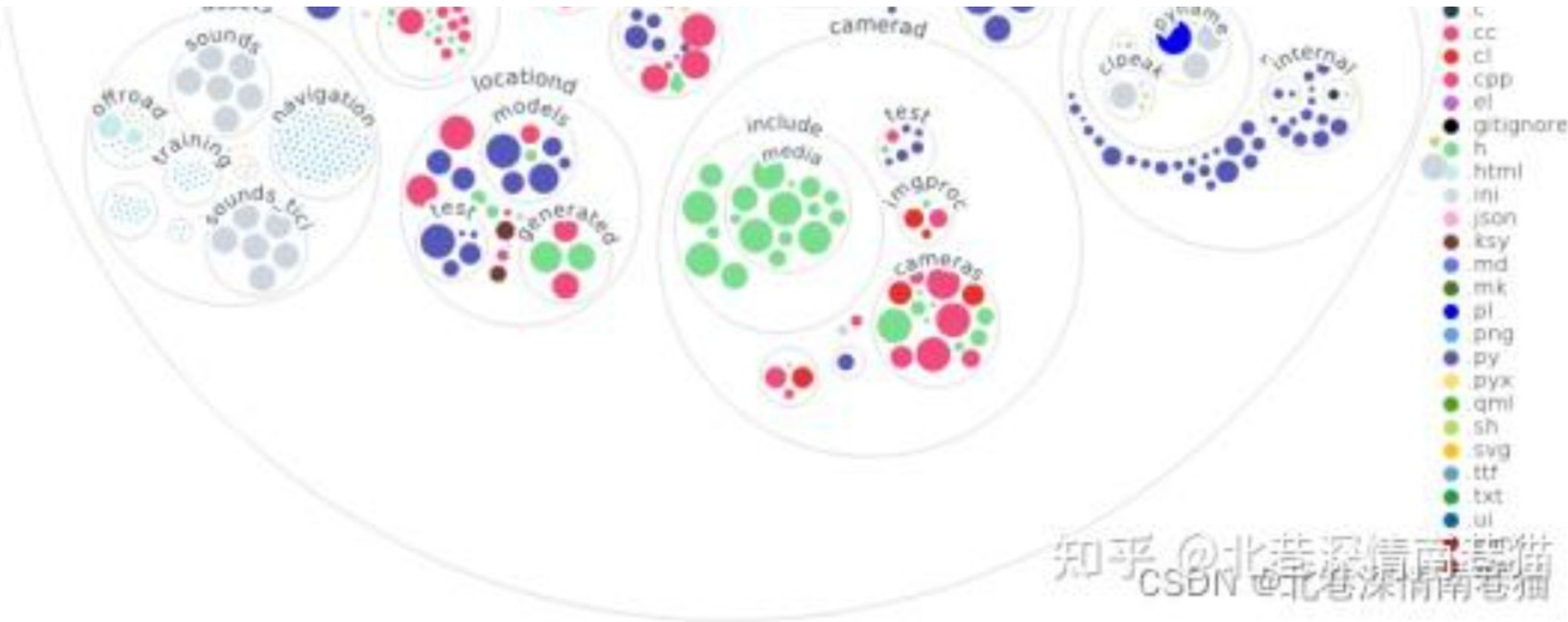
这些服务大致可分为：

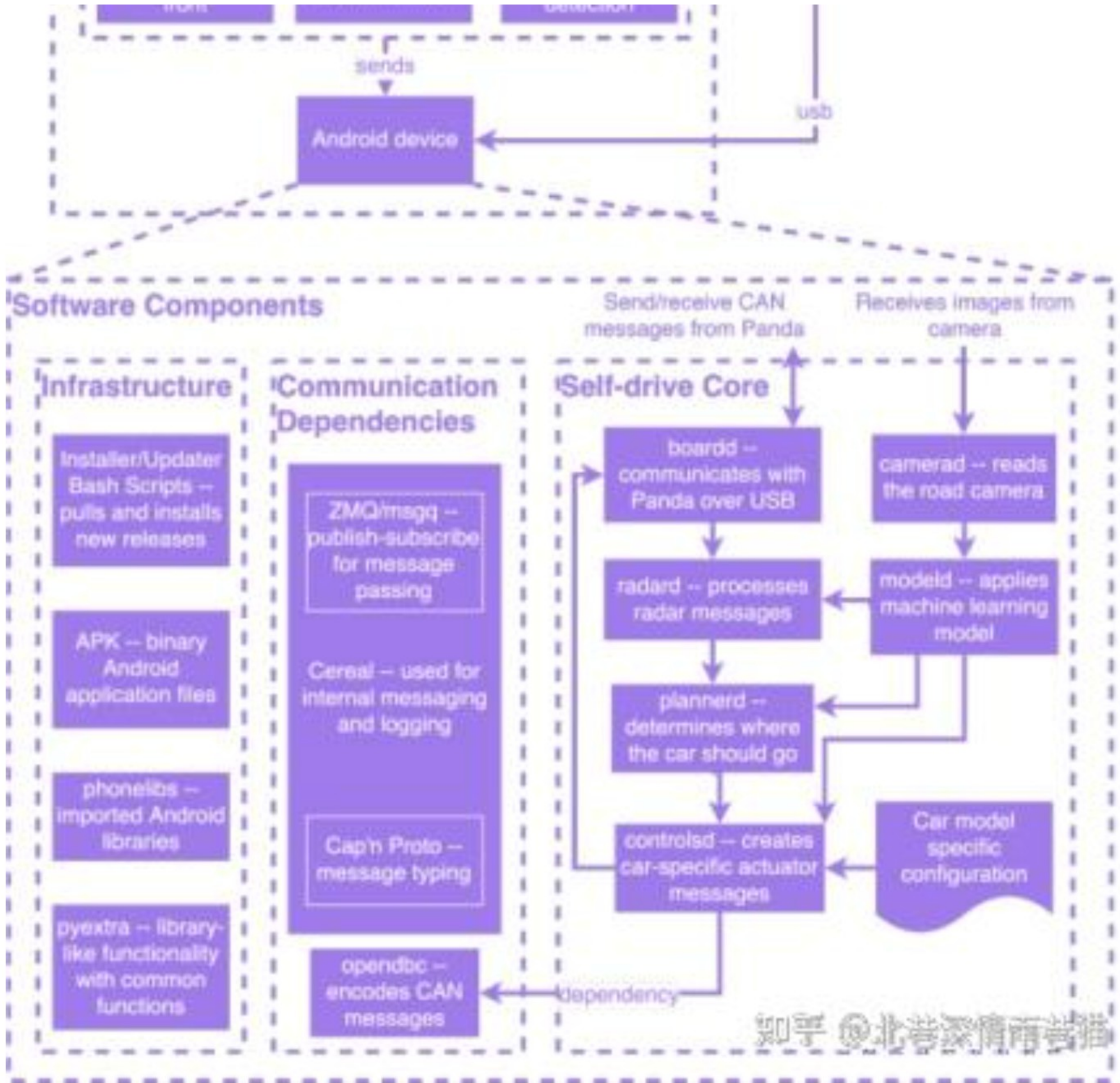
- 传感器和执行器
- 神经网络
- 定位和校准
- 控制
- 系统、日志和杂项服务



commaai/openpilot







10.汽车刹车/减速

车辆前方出现障碍物，配备ACC的汽车会检测到这一点，并且其雷达会将消息发送到CAN总线。CAN总线被panda的硬件设备连续读取。panda将此信息转发给openpilot计算单元。此计算单元（Android设备）运行多个后台进程（总共约20个），它们相互通信并协调其动作，Radard监听计算单元上发布的消息，并将注意到来自汽车雷达系统的消息。radard结合通过与由公布的数据boardd公布的雷达CAN消息modeld并发布一个radarState消息，该消息包含刹车距离等信息。radarState则被plannerd模块监听。顾名思义，规划者的责任包括规划汽车需要走的路。它发布此路径，然后其他进程可以使用它。controld模块将计划者发布的路径转换为实际的CAN消息，并且以与汽车模型无关的方式进行。

11.方向更改/道路曲线

摄像头采集环境信息，传递到核心模型，输出规划信息。从表面上看，摄像是一个非常简单的过程，唯一负责发布来自背面和正面摄像头的视频帧。这些帧然后可以被其他进程消耗。模型使用其机器学习模型对相机框架进行转换并发布结果。规划计划者监视建模的输出并将其合并到其路径规划中。plannerd会发出编码为CAN消息的方向盘扭矩变化，然后由登机板将其拾取并发送到汽车的CAN总线。

四、实车控制

1.支持的车辆品牌（截止2021-12-20）

openpilot为超过85种受支持的汽车品牌和型号执行自动车道居中和自适应巡航+控制功能。参照 comma.ai/和github.com/commaai/openpilot。

1.comma.ai supported car

Make	Model (US Market Reference)	Supported Package	ACC	No ACC accel below	No ALC below
Acura	ILX 2016-19	AcuraWatch Plus	openpilot	25mph ¹	25mph
Acura	RDX 2016-18	AcuraWatch Plus	openpilot	25mph ¹	12mph
Acura	RDX 2019-21	All	Stock	0mph	3mph
Honda	Accord 2018-21	All	Stock	0mph	3mph
Honda	Accord Hybrid 2018-21	All	Stock	0mph	3mph
Honda	Civic Hatchback 2017-21	Honda Sensing	Stock	0mph	12mph
Honda	Civic Coupe 2016-18	Honda Sensing	openpilot	0mph	12mph
Honda	Civic Coupe 2019-20	All	Stock	0mph	2mph ²
Honda	Civic Sedan 2016-18	Honda Sensing	openpilot	0mph	12mph
Honda	Civic Sedan 2019-20	All	Stock	0mph	2mph ²
Honda	CR-V 2015-16	Touring	openpilot	25mph ¹	12mph
Honda	CR-V 2017-21	Honda Sensing	Stock	0mph	12mph
Honda	CR-V Hybrid 2017-2019	Honda Sensing	Stock	0mph	12mph
Honda	e 2020	All	Stock	0mph	3mph
Honda	Fit 2018-19	Honda Sensing	openpilot	25mph ¹	12mph
Honda	Freed 2020	Honda Sensing	openpilot	25mph ¹	12mph
Honda	HR-V 2019-20	Honda Sensing	openpilot	25mph ¹	12mph
Honda	Insight 2019-21	All	Stock	0mph	3mph
Honda	Inspire 2018	All	Stock	0mph	3mph
Honda	Odyssey 2018-20	Honda Sensing	openpilot	25mph ¹	12mph
Honda	Passport 2019-21	All	openpilot	25mph ¹	12mph
Honda	Pilot 2016-21	Honda Sensing	openpilot	25mph ¹	12mph
Honda	Ridgeline 2017-21	Honda Sensing	openpilot	25mph ¹	12mph
Hyundai	Palisade 2020-21	All	Stock	0mph	0mph
Hyundai	Sonata 2020-22	All	Stock	0mph	0mph
Lexus	CT Hybrid 2017-18	LSS	Stock ³	0mph	0mph
Lexus	ES 2019-21	All	openpilot	0mph	0mph
Lexus	ES Hybrid 2017-18	LSS	Stock ³	0mph	0mph
Lexus	ES Hybrid 2019-21	All	openpilot	0mph	0mph
Lexus	IS 2017-2019	All	Stock	22mph	0mph
Lexus	NX 2018-2019	All	Stock ³	0mph	0mph
Lexus	NX 2020	All	openpilot	0mph	0mph
Lexus	NX Hybrid 2018-19	All	Stock ³	0mph	0mph

Lexus	RC 2020	All	Stock	22mph	0mph
Lexus	RX 2016-18	All	Stock ³	0mph	0mph
Lexus	RX 2020-21	All	openpilot	0mph	0mph
Lexus	RX Hybrid 2016-19	All	Stock ³	0mph	0mph
Lexus	RX Hybrid 2020-21	All	openpilot	0mph	0mph
Lexus	UX Hybrid 2019-21	All	openpilot	0mph	0mph
Toyota	Alphard 2020	All	openpilot	0mph	0mph
Toyota	Avalon 2016-21	TSS-P	Stock ³	20mph ¹	0mph
Toyota	Avalon Hybrid 2019-21	TSS-P	Stock ³	20mph ¹	0mph
Toyota	Camry 2018-20	All	Stock	0mph ⁴	0mph
Toyota	Camry 2021-22	All	openpilot	0mph	0mph

Toyota	C-HR 2017-21	All	Stock	0mph	0mph
Toyota	C-HR Hybrid 2017-19	All	Stock	0mph	0mph
Toyota	Corolla 2017-19	All	Stock ³	20mph ¹	0mph
Toyota	Corolla 2020-22	All	openpilot	0mph	0mph
Toyota	Corolla Hatchback 2019-22	All	openpilot	0mph	0mph
Toyota	Corolla Hybrid 2020-22	All	openpilot	0mph	0mph
Toyota	Highlander 2017-19	All	Stock ³	0mph	0mph
Toyota	Highlander 2020-22	All	openpilot	0mph	0mph
Toyota	Highlander Hybrid 2017-19	All	Stock ³	0mph	0mph
Toyota	Highlander Hybrid 2020-22	All	openpilot	0mph	0mph
Toyota	Mirai 2021	All	openpilot	0mph	0mph
Toyota	Prius 2016-20	TSS-P	Stock ³	0mph	0mph
Toyota	Prius 2021	All	openpilot	0mph	0mph
Toyota	Prius Prime 2017-20	All	Stock ³	0mph	0mph
Toyota	Prius Prime 2021-22	All	openpilot	0mph	0mph
Toyota	Rav4 2016-18	TSS-P	Stock ³	20mph ¹	0mph
Toyota	Rav4 2019-21	All	openpilot	0mph	0mph
Toyota	Rav4 Hybrid 2016-18	TSS-P	Stock ³	0mph	0mph
Toyota	Rav4 Hybrid 2019-21	All	openpilot	0mph	0mph
Toyota	Sienna 2018-20	All	Stock ³	0mph	0mph

2.Community Maintained Cars and Features

Make	Model (US Market Reference)	Supported Package	ACC	No ACC accel below	No ALC below
Audi	A3 2014-19	ACC + Lane Assist	Stock	0mph	0mph
Audi	A3 Sportback e-tron 2017-18	ACC + Lane Assist	Stock	0mph	0mph
Audi	Q2 2018	ACC + Lane Assist	Stock	0mph	0mph
Audi	Q3 2020-21	ACC + Lane Assist	Stock	0mph	0mph
Audi	S3 2015	ACC + Lane Assist	Stock	0mph	0mph
Buick	Regal 2018 ¹	Adaptive Cruise	openpilot	0mph	7mph
Cadillac	ATS 2018 ¹	Adaptive Cruise	openpilot	0mph	7mph
Cadillac	Escalade ESV 2016 ¹	ACC + LKAS	openpilot	0mph	7mph
Chevrolet	Malibu 2017 ¹	Adaptive Cruise	openpilot	0mph	7mph
Chevrolet	Volt 2017-18 ¹	Adaptive Cruise	openpilot	0mph	7mph
Chrysler	Pacifica 2017-18	Adaptive Cruise	Stock	0mph	9mph
Chrysler	Pacifica 2020	Adaptive Cruise	Stock	0mph	39mph
Chrysler	Pacifica Hybrid 2017-18	Adaptive Cruise	Stock	0mph	9mph
Chrysler	Pacifica Hybrid 2019-21	Adaptive Cruise	Stock	0mph	39mph
Genesis	G70 2018	All	Stock	0mph	0mph
Genesis	G70 2020	All	Stock	0mph	0mph
Genesis	G80 2018	All	Stock	0mph	0mph
Genesis	G90 2018	All	Stock	0mph	0mph
GMC	Acadia 2018 ¹	Adaptive Cruise	openpilot	0mph	7mph
Holden	Astra 2017 ¹	Adaptive Cruise	openpilot	0mph	7mph
Hyundai	Elantra 2017-19	SCC + LKAS	Stock	19mph	34mph

Hyundai	Ioniq Hybrid 2020-22	SCC + LFA	Stock	0mph	0mph
Hyundai	Ioniq PHEV 2020-21	SCC + LKAS	Stock	0mph	0mph
Hyundai	Kona 2020	SCC + LKAS	Stock	0mph	0mph
Hyundai	Kona EV 2018-19	SCC + LKAS	Stock	0mph	0mph
Hyundai	Kona Hybrid 2020	SCC + LKAS	Stock	0mph	0mph
Hyundai	Santa Fe 2019-20	All	Stock	0mph	0mph
Hyundai	Santa Fe 2021-22	All	Stock	0mph	0mph
Hyundai	Santa Fe Hybrid 2022	All	Stock	0mph	0mph
Hyundai	Sonata 2018-2019	SCC + LKAS	Stock	0mph	0mph
Hyundai	Sonata Hybrid 2021-22	All	Stock	0mph	0mph
Hyundai	Veloster 2019-20	SCC + LKAS	Stock	5mph	0mph
Jeep	Grand Cherokee 2016-18	Adaptive Cruise	Stock	0mph	9mph
Jeep	Grand Cherokee 2019-20	Adaptive Cruise	Stock	0mph	39mph
Kia	Ceed 2019	SCC + LKAS	Stock	0mph	0mph
Kia	Forte 2018-21	SCC + LKAS	Stock	0mph	0mph
Kia	K5 2021-22	SCC + LFA	Stock	0mph	0mph
Kia	Niro EV 2019-21	SCC + LKAS	Stock	0mph	0mph

Kia	Niro Hybrid 2021	SCC + LKAS	Stock	0mph	0mph
Kia	Niro PHEV 2019	SCC + LKAS	Stock	10mph	32mph
Kia	Optima 2017	SCC + LKAS	Stock	0mph	32mph
Kia	Optima 2019	SCC + LKAS	Stock	0mph	0mph
Kia	Seltos 2021	SCC + LKAS	Stock	0mph	0mph

Kia	Sorento 2018-19	SCC + LKAS	Stock	0mph	0mph
Kia	Stinger 2018	SCC + LKAS	Stock	0mph	0mph
Kia	Telluride 2020	SCC + LKAS	Stock	0mph	0mph
Mazda	CX-9 2021	All	Stock	0mph	28mph
Nissan	Altima 2019-20	ProPILOT	Stock	0mph	0mph
Nissan	Leaf 2018-22	ProPILOT	Stock	0mph	0mph
Nissan	Rogue 2018-20	ProPILOT	Stock	0mph	0mph
Nissan	X-Trail 2017	ProPILOT	Stock	0mph	0mph
SEAT	Ateca 2018	Driver Assistance	Stock	0mph	0mph
SEAT	Leon 2014-2020	Driver Assistance	Stock	0mph	0mph
Škoda	Kamiiq 2021 ²	Driver Assistance	Stock	0mph	0mph
Škoda	Karoq 2019	Driver Assistance	Stock	0mph	0mph
Škoda	Kodiaq 2018-19	Driver Assistance	Stock	0mph	0mph
Škoda	Octavia 2015, 2018-19	Driver Assistance	Stock	0mph	0mph
Škoda	Octavia RS 2016	Driver Assistance	Stock	0mph	0mph
Škoda	Scala 2020	Driver Assistance	Stock	0mph	0mph
Škoda	Superb 2015-18	Driver Assistance	Stock	0mph	0mph
Subaru	Ascent 2019	EyeSight	Stock	0mph	0mph
Subaru	Crosstrek 2018-20	EyeSight	Stock	0mph	0mph
Subaru	Forester 2019-21	EyeSight	Stock	0mph	0mph
Subaru	Impreza 2017-19	EyeSight	Stock	0mph	0mph
Volkswagen	Arteon 2021 ⁴	Driver Assistance	Stock	0mph	0mph
Volkswagen	Atlas 2018-19	Driver Assistance	Stock	0mph	0mph

Volkswagen	California 2021 ⁴	Driver Assistance	Stock	0mph	32mph
Volkswagen	e-Golf 2014, 2019-20	Driver Assistance	Stock	0mph	0mph
Volkswagen	Golf 2015-20	Driver Assistance	Stock	0mph	0mph
Volkswagen	Golf Alltrack 2017-18	Driver Assistance	Stock	0mph	0mph
Volkswagen	Golf GTE 2016	Driver Assistance	Stock	0mph	0mph
Volkswagen	Golf GTI 2018-20	Driver Assistance	Stock	0mph	0mph
Volkswagen	Golf R 2016-19	Driver Assistance	Stock	0mph	0mph
Volkswagen	Golf SportsVan 2016	Driver Assistance	Stock	0mph	0mph
Volkswagen	Golf SportWagen 2015	Driver Assistance	Stock	0mph	0mph
Volkswagen	Jetta 2018-20	Driver Assistance	Stock	0mph	0mph
Volkswagen	Jetta GLI 2021	Driver Assistance	Stock	0mph	0mph
Volkswagen	Passat 2016-18 ³	Driver Assistance	Stock	0mph	0mph
Volkswagen	Polo 2020	Driver Assistance	Stock	0mph	0mph
Volkswagen	T-Cross 2021 ⁴	Driver Assistance	Stock	0mph	0mph
Volkswagen	T-Roc 2021 ⁴	Driver Assistance	Stock	0mph	0mph
Volkswagen	Taos 2022 ⁴	Driver Assistance	Stock	0mph	0mph
Volkswagen	Tiguan 2020	Driver Assistance	Stock	0mph	0mph
Volkswagen	Touran 2017	Driver Assistance	Stock	0mph	0mph

2.实车要求配备

首要条件需要具备两种系统：ACC主动式定速巡航系统⁺与辅助转向⁺或相似系统。首先需要控制汽车要确认可以控制汽车油门、刹车及方向盘，而这两个系统刚好可以证实一辆汽车是可控制的。

ACC主动式定速巡航系统：可以确认油门即刹车

辅助转系统：可以确认方向盘

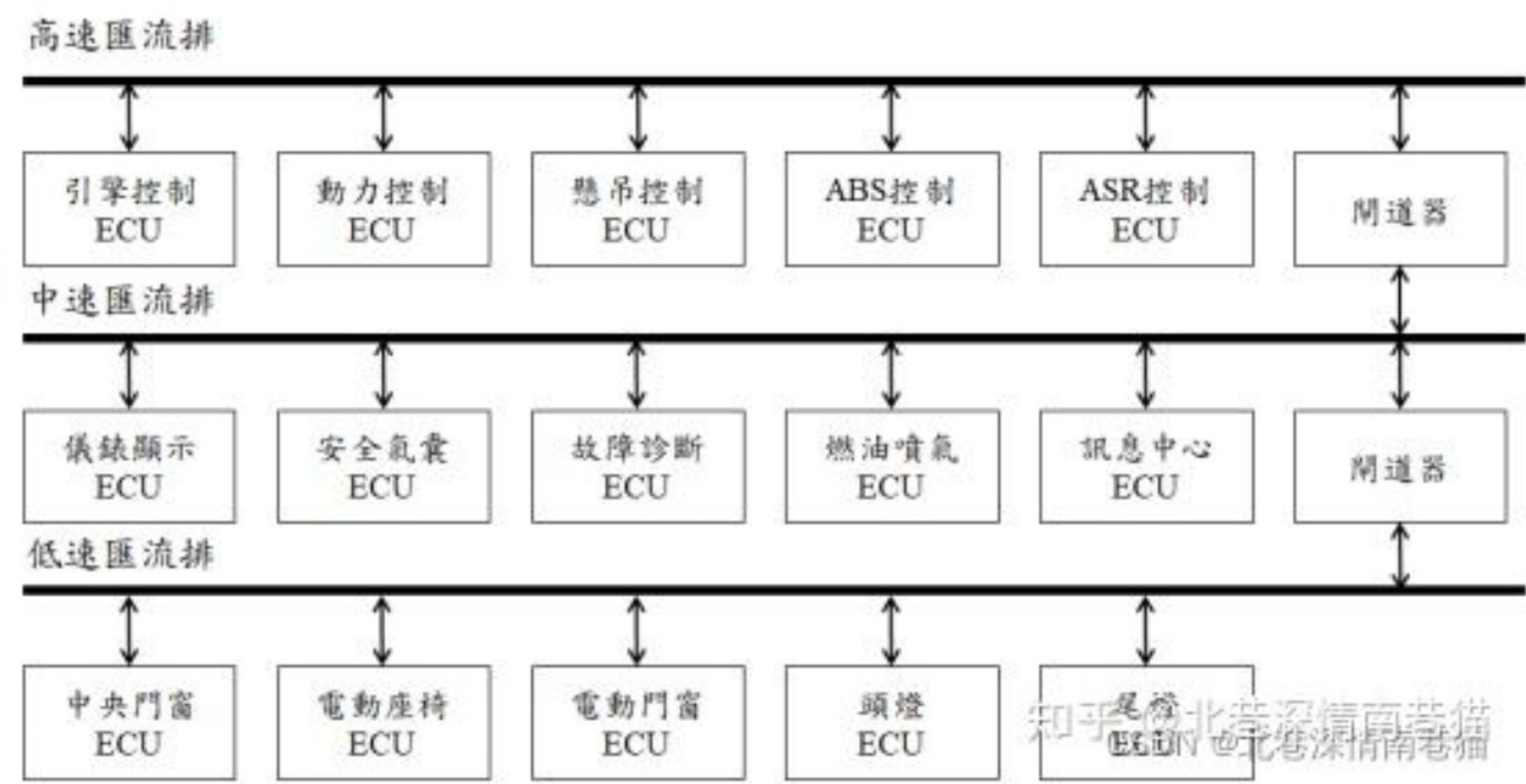
通过Openpilot github的介绍就会发现，他们官方认证的车款都是具有Honda Sensing 或 Toyota Safety Sense-P(简称TSS-P)。因为当他们已经破解了一台具备TSS-P标准的汽车，那要移植到另一辆车种但同样具备TSS-P标准上时轻松许多、code也只需要变动少部份。最后再次强调Honda Sensing 与 TSS-P的标准都是拥有ACC与转向辅助系统⁺、如下图所示。



3.CAN总线

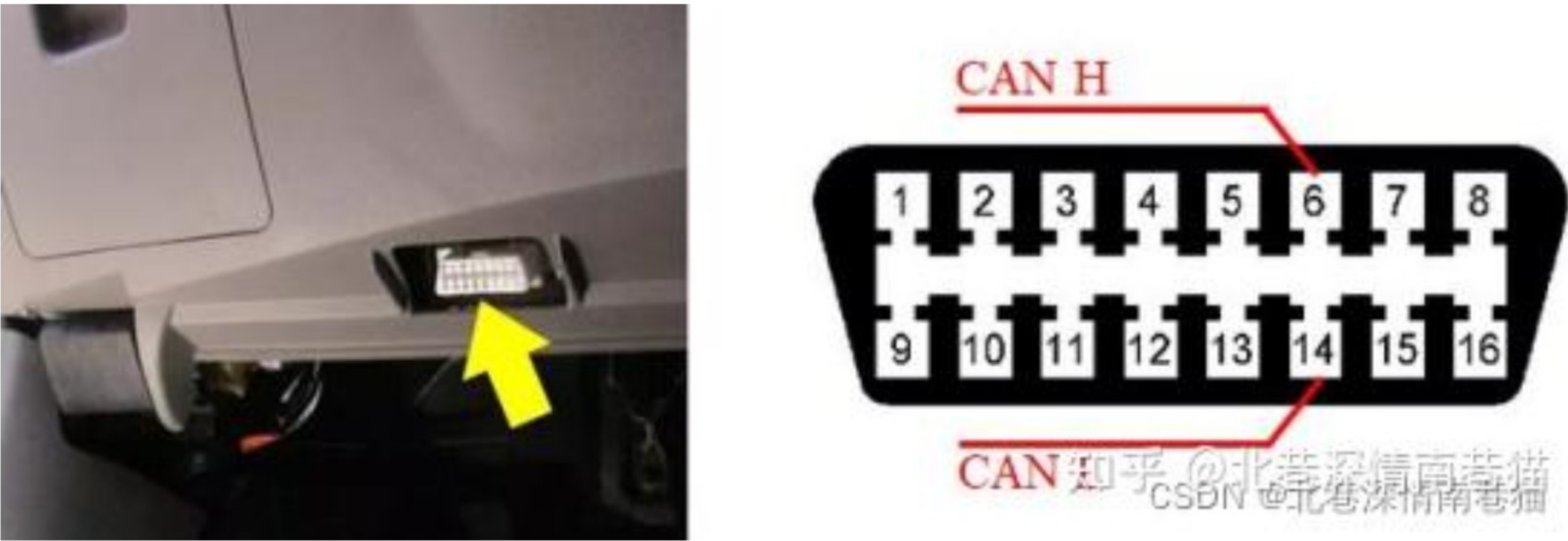
现在绝大多数的汽车内部都拥有超过50个以上的ECU原件，而ECU之间的通讯绝大多数都是使用CAN buses的标准协定，而CAN bus又可以分为高数与低速，如图，所以这也代表我们有机会透过

CANbus协定来读取或传值给ECU。



4.汽车CAN bus连接方法

现在几乎所有汽车都拥有OBDII，如图。经由OBDII 接口可以连接至汽车内部CANbus網路，如图。到这步你可以读取一些汽车讯息，例如：车速、车门状态等车辆资讯，但还没办法控制油门刹车等等。



5.汽车CAN bus预留接口

前面有稍为提及到汽车网路是由CAN bus所组成，但只答对了一半，实际上汽车内部网路大概会像图六这样，可能由多组CAN bus或其他协定所组成，所以光凭OBDII 接口是无法接管整辆汽车的，为此我们需要找出更多的通讯接口。很幸运的许多高智能汽车，都会有一个Forward Recognition Camera于照后镜盖中，如图。FRC的其一功能就是侦测车到并控制方向盘防止道路偏移，所以经由这个部位能够个深入汽车内部并接管汽车方向盘、油门、刹车等等。



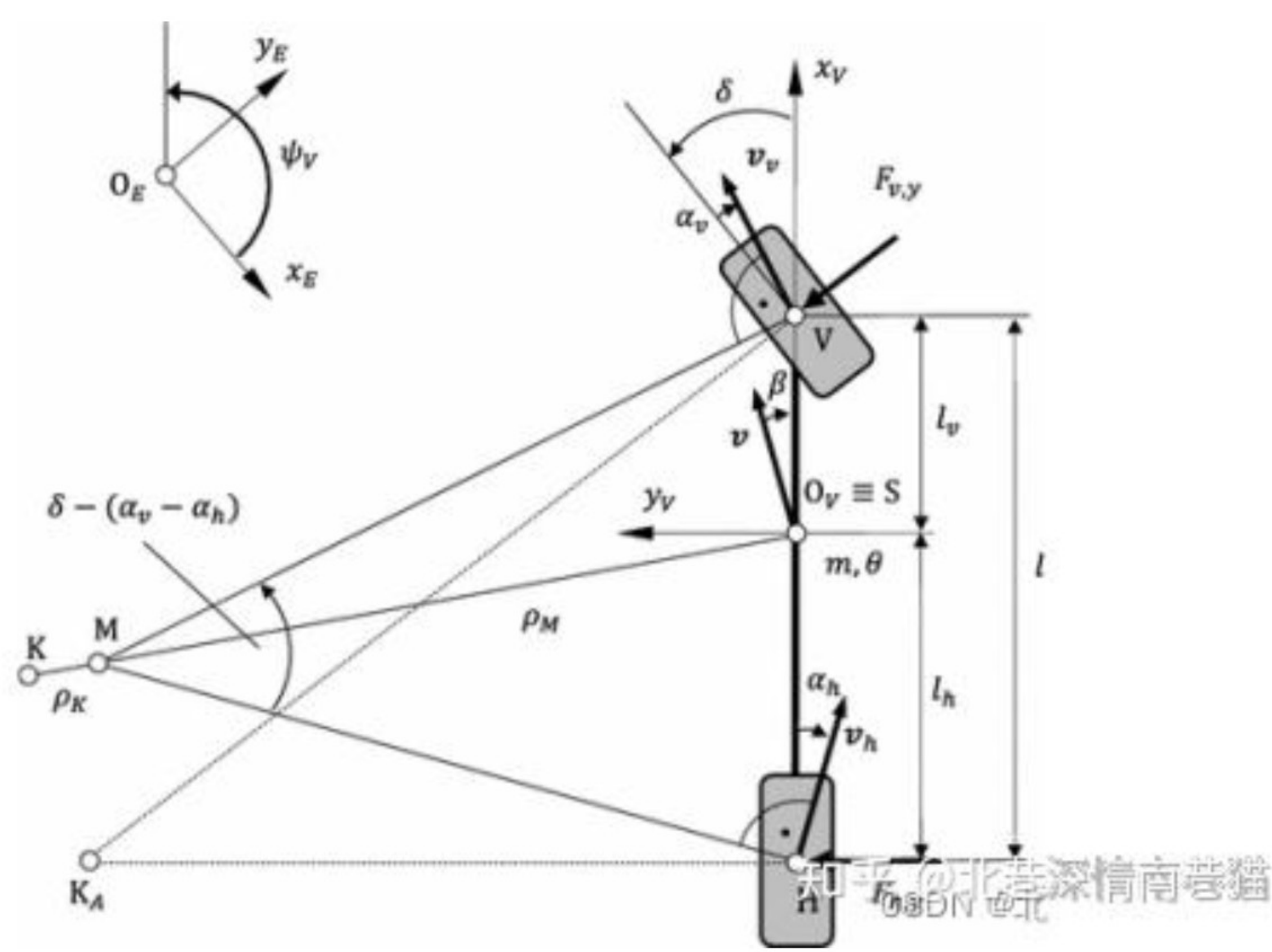
下图为Comma ai 提供的Panda转接板。



五、Openpilot关键技术

1.汽车模型

该项目通过估计有关汽车的一些关键变量来使转弯更加准确，从而改善了驾驶体验。openpilot从模型接收车道线的位置和所需的路径。使用模型预测控制⁺，可以从汽车的当前位置到所需路径生成平滑的轨迹。要实际行驶这条路，我们必须计算出方向盘转动多少。为了将所需的轨迹转换成转向角，使用了汽车的数学模型。使用该模型，我们可以根据转向角和车辆速度来预测汽车将转弯多少。

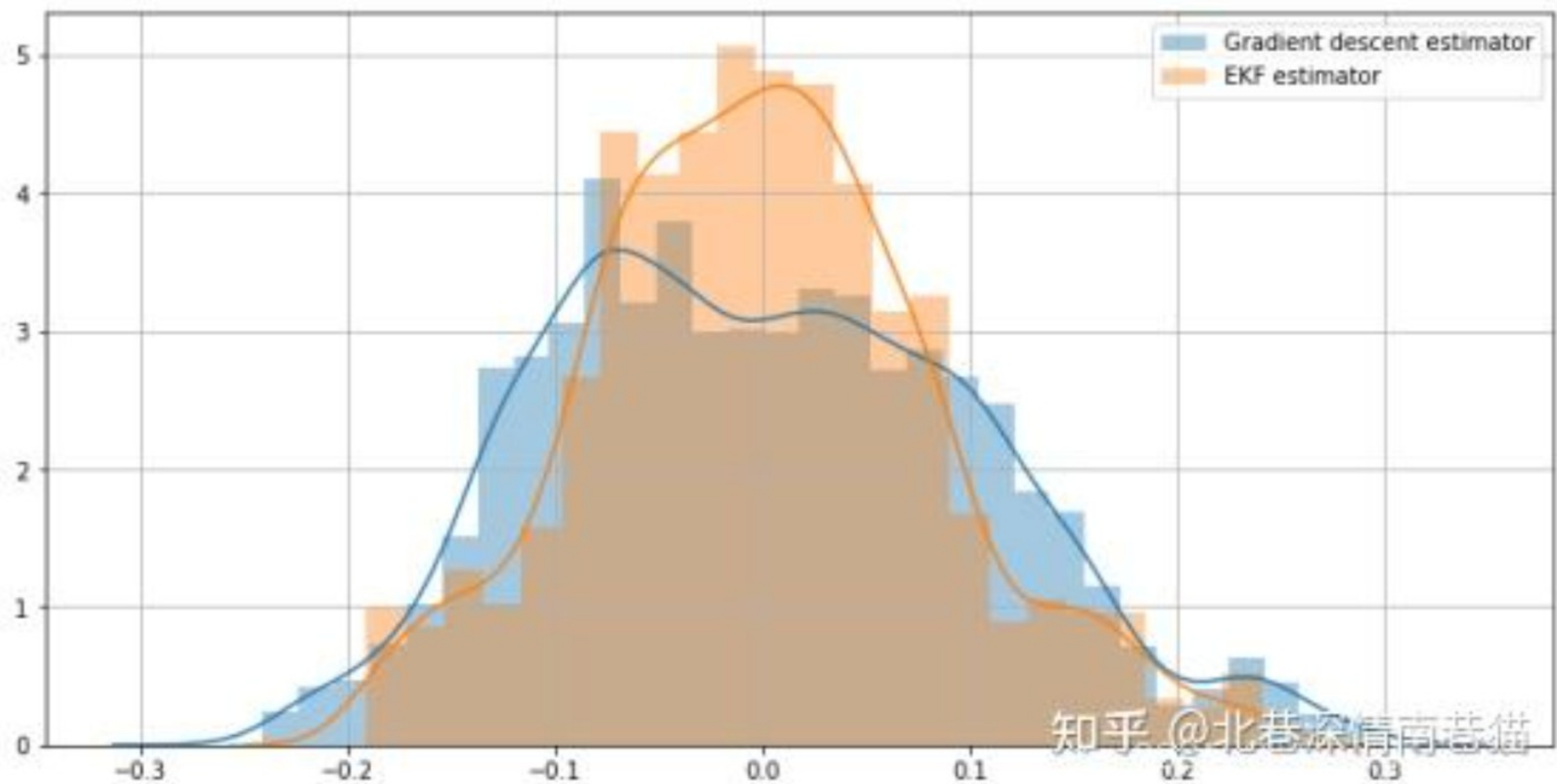


图片导入失败, 请重新上传

我们发现未正确安装的对准或转向角传感器*存在许多错误。当openpilot认为方向盘是直的时，这会导致汽车转弯。同样，道路也不是完全平坦的，特别是在急转弯时，道路常常是倾斜的。当方向盘笔直时，这会使汽车漂移到道路的下部。这两个误差都可以建模为测得的转向角和实际转向角之间的偏移量，我们称之为角度偏移量。我们将其分解为两个单独的变量，即平均角度偏移量（长期影响，如路线）和快速角度偏移量（短期影响，如路堤）。下图显示了行驶在非常倾斜的转弯处的汽车一分钟段的（估计）角度偏移量的曲线图。



为了确保汽车遵循期望的轨迹，我们要不断改进这两个角度偏移的估计。在行驶过程中，我们可以使用车辆模型来预测在当前转向角和速度下汽车应转弯多少。然后，我们可以使用我们的精确定位器来测量汽车实际转弯的程度，并使用它来改进参数的估算。为了估计参数，我们使用扩展卡尔曼滤波器（EKF），因为车辆模型的方程是非线性的。研究小组已经编写了一个非常不错的EKF库 rednose。使用SymPy，我们可以定义控制车辆动力学的方程式，并且rednose能自动生成高性能C++代码以实际运行估算。实施此方法并依次比较车道中心之间的距离后，我们发现以前的基于梯度下降的算法（蓝色）和基于新的EKF的算法（橙色）之间的转弯精度有了显著提高。基于倾斜度下降的倾斜度每转都偏向内部或外部，而EKF无偏。



2.Openpilot安全模型

安全模型具有三个主要原则：

- 驾驶员必须始终保持关注；
- 驾驶员必须始终能够立即恢复对车辆的手动控制；
- 车辆不得过快改变其轨迹，以使驾驶员安全地做出反应。

遵守原则（1）的规定显然取决于驾驶者，但是我们遵循了[行业标准](#)惯例，并构建了基于摄像头的例如通用[超级巡航](#)的驾驶者监控系统，和特斯拉 Autopilot 的“手不离[方向盘](#)”的检测器，来帮助驾驶者保持专注。该代码位于 `driver_monitor.py` 中，只要您不通过降低严格性来禁用或削弱它，就表示您符合原则（1）。虽然原厂的 openpilot 同时使用这两种方式，但只要特斯拉可以使用，我们就可以接受其中一种。只是两者都不是。

原则（2）由[熊猫](#)中的安全代码强制执行，这是我们到汽车的实时STM4桥接器。熊猫具有状态变量“`controls_allowed`”，该变量确定是否允许在CAN总线上发送控制消息。您可以通过打开[巡航控制](#)来进入控制允许状态，并通过取消巡航控制来退出。**制动踏板必须始终立即取消控制允许状态。**在开放式自动驾驶仪中，油门踏板也总是会取消的，尽管大巡航中有一个不安全的标志允许熊猫在加油时使用，因为超级巡航和自动驾驶仪都允许这样做。

原则（3）是最微妙的。汽车可以压倒人类，我们需要确保人类始终处于控制之中。通过使用为[ADAS](#)设计的CAN消息，我们从汽车的内置安全模型中获得了很多保护。**请勿使用不是为ADAS设计的消息或ADAS规范之外的消息。**此外，在进行注入测试之后，我们在熊猫中编写了额外的安全层，以限制如何使用这些消息。

请记住，在2级系统中，**不执行任何操作始终是安全的选择。您绝不能依靠汽车采取或保持动作，只能依靠它来避免踩刹车或在踩刹车后继续动作。**

该安全模型的优点在于，**openpilot的功能安全性都不依赖于神经网络，甚至不依赖于EON上运行的任何东西。**因此，可以随意使用模型，UI，调整，控件，设备硬件或传感器。不用理会熊猫代码和驾驶员监控，而整体安全是一个整体，但功能安全性将通过许多不同的修改而保持不变。

3.深度学习模型

Openpilot项目中存在两种属性模型，分别是驾驶模型（supercombo）和驾驶员监控模型（dmonitoring_model）。

 dmonitoring_model.current	wider FOV DM on comma three (#22286)
 dmonitoring_model.onnx	wider FOV DM on comma three (#22286)
 dmonitoring_model.q.dlc	wider FOV DM on comma three (#22286)
 supercombo.dlc	New model: 224877e7-dfcf-4b28-9482-371655918fe1/950 (#23170)
 supercombo.onnx	New model: 224877e7-dfcf-4b28-9482-371655918fe1/950 (#23170)

其中，dlc格式模型，适用于高通加速器终端使用，onnx格式为通用格式，不限制平台和终端，模型可视化工具⁺为netron⁺。具体参考：[github.com/commaai/open...](https://github.com/commaai/openpilot)

1.Supercombo模型

Supercombo input format (Full size: 393738 x float32)

- image stream
 - Two consecutive images (256 * 512 * 3 in RGB) recorded at 20 Hz : 393216 = 2 * 6 * 128 * 256
 - Each 256 * 512 image is represented in YUV420 with 6 channels : 6 * 128 * 256
 - Channels 0,1,2,3 represent the full-res Y channel and are represented in numpy as Y[:,2, ::2], Y[:,2, 1::2], Y[1::2, ::2], and Y[1::2, 1::2]
 - Channel 4 represents the half-res U channel
 - Channel 5 represents the half-res V channel
- desire
 - one-hot encoded vector to command model to execute certain actions, bit only needs to be sent for 1 frame : 8
- traffic convention
 - one-hot encoded vector to tell model whether traffic is right-hand or left-hand traffic : 2
- recurrent state
 - The recurrent state vector that is fed back into the GRU for temporal context : 512

Supercombo output format (Full size: 6472 x float32)

- plan
 - 5 potential desired plan predictions : 4955 = 5 * 991
 - predicted mean and standard deviation of the following values at 33 timesteps : 990 = 2 * 33 * 15
 - x,y,z position in current frame (meters)
 - x,y,z velocity in local frame (meters/s)
 - x,y,z acceleration local frame (meters/(s*s))
 - roll, pitch , yaw in current frame (radians)
 - roll, pitch , yaw rates in local frame (radians/s)
 - probability^[1] of this plan hypothesis being the most likely: 1
- lanelines
 - 4 lanelines (outer left, left, right, and outer right): 528 = 4 * 132
 - predicted mean and standard deviation for the following values at 33 x positions : 132 = 2 * 33 * 2
 - y position in current frame (meters)
 - z position in current frame (meters)

- laneline probabilities

- 2 probabilities^[1] that each of the 4 lanelines exists : $8 = 4 * 2$
 - deprecated probability
 - used probability

- road-edges

- 2 road-edges (left and right): $264 = 2 * 132$
 - predicted mean and standard deviation for the following values at 33 x positions : $132 = 2 * 33 * 2$
 - y position in current frame (meters)
 - z position in current frame (meters)

- leads

- 2 hypotheses for potential lead cars : $102 = 2 * 51$
 - predicted mean and stadard deviation for the following values at 0,2,4,6,8,10s : $48 = 2 * 6 * 4$
 - x position of lead in current frame (meters)
 - y position of lead in current frame (meters)
 - speed of lead (meters/s)
 - acceleration of lead(meters/(s*s))
 - probabilities^[1] this hypothesis is the most likely hypothesis at 0s, 2s or 4s from now : 3

- lead probabilities

- probability^[1] that there is a lead car at 0s, 2s, 4s from now : $3 = 1 * 3$

- desire state

- probability^[1] that the model thinks it is executing each of the 8 potential desires at 0,2,4,6,8,10s : $8 = 1 * 8$

- meta ^[2]

- Various metadata about the scene : $80 = 1 + 35 + 12 + 32$
 - Probability^[1] that openpilot is engaged : 1
 - Probabilities^[1] of various things happening between now and 2,4,6,8,10s : $35 = 5 * 7$
 - Disengage of openpilot with gas pedal
 - Disengage of openpilot with brake pedal
 - Override of openpilot steering
 - 3m/(s*s) of deceleration
 - 4m/(s*s) of deceleration
 - 5m/(s*s) of deceleration
 - Probabilities^[1] of left or right blinker being active at 0,2,4,6,8,10s : $12 = 6 * 2$
 - Probabilities^[1] that each of the 8 desires is being executed at 0,2,4,6s : $32 = 4 * 8$

- pose ^[2]

- predicted mean and standard deviation of current translation and rotation rates : $12 = 2 * 6$
 - x,y,z velocity in current frame (meters/s)
 - roll, pitch , yaw rates in current frame (radians/s)
- recurrent state
 - The recurrent state vector that is fed back into the GRU for temporal context : 512

2.Driver Monitoring Model

- .onnx model can be run with onnx runtimes
- .dlc file is a pre-quantized model and only runs on qualcomm DSPs

input format

- single image (640 * 320 * 3 in RGB):
 - full input size is $6 * 640/2 * 320/2 = 307200$
 - represented in YUV420 with 6 channels:
 - Channels 0,1,2,3 represent the full-res Y channel and are represented in numpy as $Y[:,2, :2]$, $Y[:,2, 1:2]$, $Y[1:2, :2]$, and $Y[1:2, 1:2]$
 - Channel 4 represents the half-res U channel
 - Channel 5 represents the half-res V channel
 - normalized, ranging from -1.0 to 1.0

output format

- 39 x float32 outputs (parsing example)
 - face pose: $12 = 6 + 6$
 - face orientation [pitch, yaw, roll] in camera frame: 3
 - face position [dx, dy] relative to image center: 2
 - normalized face size: 1
 - standard deviations for above outputs: 6
 - face visible probability: 1
 - eyes: $20 = (8 + 1) + (8 + 1) + 1 + 1$
 - eye position and size, and their standard deviations: 8
 - eye visible probability: 1
 - eye closed probability: 1
 - wearing sunglasses probability: 1
 - poor camera vision probability: 1
 - face partially out-of-frame probability: 1
 - (deprecated) distracted probabilities: 2
 - face covered probability: 1

4.开源数据集

1.Comma2K19

官方开源了comma2k19数据集，记录了加州280高速公路上超过33小时视频。有2019段，每段1分钟，在加利福尼亚圣若泽和旧金山之间的高速公路上行驶20公里。comma2k19是一个完全可复制和可扩展的数据集。这些数据是使用comma EONs收集的，它的传感器类似于任何现代智能手机，包括面向道路的摄像头、手机GPS、温度计和9轴IMU。此外，EON使用panda捕捉原始GNSS值和汽车发送的所有CAN数据。



在这里，我们还介绍了Laika，一个开放源码的GNSS处理库。与用于收集原始数据的[全球导航卫星系统+模块](#)相比，Laika的定位精度提高了40%。该数据集包括记录相机的全局参考帧中的姿势（位置+方向）估计。这些姿态是通过紧密耦合的INS/GNSS/Vision优化器计算的，该优化器依赖于Laika处理的数据。comma2k19是开发和验证紧密耦合的[GNSS算法+](#)和用于商品传感器的映射算法的理想选择。



数据集结构：

```
Dataset_chunk_n
|
+-- route_id (dongle_id|start_time)
|
+-- segment_number
|
+-- preview.png (first frame video)
+-- raw_log.bz2 (raw capnp log, can be read with openpilot-tools: logreader)
+-- video.hevc (video file, can be read with openpilot-tools: framereader)
+-- processed_log/ (processed logs as numpy arrays, see format for detail)
+-- global_pos/ (global poses of camera as numpy arrays, see format for detail)
```



```

processed_log
|
|--IMU ([forward, right, down])
| |
| | ++acceleration: (m^2/s)
| | ++gyro_uncalibrated (rad/s)
| | ++gyro_bias: android gyro bias estimate (rad/s)
| | ++gyro: with android bias correction (rad/s)
| | ++magnetic_uncalibrated: (T)
| | ++magnetic: with android calibration(T)
|
|--CAN data:
| |
| | ++car_speed (m/s)
| | ++steering_angle (deg)
| | ++wheel_speeds: [front_left, front_right, rear_left, rear_right] (m/s)
| | ++radar: [forward distance (m),
| |           left distance (m),
| |           relative speed (m/s),
| |           nan,
| |           nan,
| |           address,
| |           new_track (bool)]
| | ++raw CAN: This not stored as a value array but as three seperate arrays [src, address, data]
|
|--GNSS
| |
| | ++live_gnss_qcom: [latitude (deg),
| |                   longitude (deg),
| |                   speed (m/s),
| |                   utc_timestamp (s),
| |                   altitude (m),
| |                   bearing (deg)]
| | ++live_gnss_ublox: [latitude (deg),
| |                     longitude (deg),
| |                     speed (m/s),
| |                     utc_timestamp (s),
| |                     altitude (m),
| |                     bearing (deg)]

```

知乎 @北巷深情南巷猫
CSDN @北巷深情南巷猫

Pose Format

The poses of the camera and timestamps of every frame of the video are stored as follows:
frame_times: timestamps of video frames in boot time (s)
frame_gps_times: timestamps of video frames in gps_time: ([gps week (weeks), time-of-week (s)])
frame_positions: global positions in ECEF of camera(m)
frame_velocities: global velocity in ECEF of camera (m/s)
frame_orientations: global orientations as quaternion needed to rotate from ECEF frame to local camera frame defined as [forward, right, down] (hamilton quaternion)

知乎 @北巷深情南巷猫
CSDN @北巷深情南巷猫

2.Comma10K



目录：

```
imgs/ -- The PNG image files
masks/ -- PNG segmentation masks (update these!)
imgs2/ -- New PNG image files paired with fisheye PNGs
masks2/ -- PNG segmentation masks (update these!)
segs/ -- The outputs in probability from our internal segnet (unreleased, too big)
```

分类：

```
1 - #402020 - road (all parts, anywhere nobody would look at you funny for driving)
2 - #ff0000 - lane markings (don't include non lane markings like turn arrows and crosswalks)
3 - #808080 - undrivable
4 - #00ff66 - movable (vehicles and people/animals)
5 - #cc00ff - my car (and anything inside it, including wires, mounts, etc. No reflections)
```